Vous : utilisation de QLabel avec PyQt6

Gemini a dit

Le QLabel est l'un des widgets les plus fondamentaux de PyQt6. Bien que son nom suggère qu'il ne sert qu'à afficher du texte statique, il est en réalité très polyvalent : il peut afficher du texte enrichi (HTML), des images, des animations (GIF) et même des liens cliquables.

Voici un guide pour maîtriser son utilisation.

---

1. Utilisation de base (Texte)

Pour afficher du texte simple, il suffit d'instancier la classe et d'utiliser la méthode setText().

Python

```
from PyQt6.QtWidgets import QLabel, QWidget, QVBoxLayout


# Création du label
label = QLabel("Bonjour tout le monde !")


# Modification du texte après coup
label.setText("Nouveau message")


# Récupérer le texte
contenu = label.text()
```

---

2. Formatage et Alignement

Vous pouvez modifier l'apparence du texte en utilisant du **HTML** ou les constantes d'alignement de Qt.

Styles et HTML

Le QLabel interprète nativement les balises HTML de base.

- label.setText("<h1>Titre</h1>
<font color='red'>Texte en rouge</font>")

Alignement

Par défaut, le texte est aligné à gauche et centré verticalement.

- **Horizontal :** Qt.AlignmentFlag.AlignLeft, AlignRight, AlignCenter, AlignJustify.
- **Vertical :** Qt.AlignmentFlag.AlignTop, AlignBottom, AlignVCenter.

Python

```
from PyQt6.QtCore import Qt


label.setAlignment(Qt.AlignmentFlag.AlignCenter | Qt.AlignmentFlag.AlignVCenter)
```

---

3. Affichage d'images (QPixmap)

Le QLabel est le conteneur standard pour afficher des images dans une interface.

Python

```
from PyQt6.QtGui import QPixmap


label_image = QLabel()

pixmap = QPixmap("mon_image.jpg")
```

Optionnel : Redimensionner l'image pour qu'elle s'adapte au label

label_image.setPixmap(pixmap)

label_image.setScaledContents(True)

---

4. Fonctionnalités Avancées

Fonctionnalité	Méthode / Approche
Retours à la ligne	label.setWordWrap(True) (permet au texte de passer à la ligne si le widget est trop étroit).
Interactivité	label.setOpenExternalLinks(True) (rend les liens <a> cliquables dans le navigateur).
Raccourcis clavier	label.setBuddy(mon_input) (permet de mettre le focus sur un champ de saisie via un raccourci défini dans le label).
Animations	Utiliser QMovie avec label.setMovie(movie) pour afficher des GIFs.

Exemple Complet : Une interface simple

Python

import sys

from PyQt6.QtWidgets import QApplication, QWidget, QVBoxLayout, QLabel

from PyQt6.QtCore import Qt

```
class MaFenetre(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Démo QLabel")
        layout = QVBoxLayout()

        # Label avec Style CSS
        titre = QLabel("Bienvenue sur PyQt6")
        titre.setStyleSheet("font-size: 20px; color: blue; font-weight: bold;")
        titre.setAlignment(Qt.AlignmentFlag.AlignCenter)

        # Label avec Word Wrap
        description = QLabel("Ceci est un texte très long qui va automatiquement passer à la ligne grâce à la propriété WordWrap.")
        description.setWordWrap(True)

        layout.addWidget(titre)
        layout.addWidget(description)
        self.setLayout(layout)

app = QApplication(sys.argv)
```

```
fenetre = MaFenetre()

fenetre.show()

sys.exit(app.exec())
```

Souhaitez-vous que je vous montre comment personnaliser davantage l'apparence des labels avec des feuilles de style (QSS) pour créer des bordures ou des arrondis ?

Vous avez dit

Corrigez le code suivant : from PyQt6.QtWidgets import QMainWindow, QLabel, QVBoxLayout, QPushButton, QHBoxLayout, QWidget

```
from PyQt6.QtGui import QIcon

from PyQt6.QtCore import Qt

import os


class App(QMainWindow):

    def __init__(self):

        super().__init__()

        self.setup_window()

        self.load_icon()

        self.load_style()

        self.add_Layouts()


        self.valeur_label = 0
```

```
# ----------------------
# Configuration de la fenêtre
# ----------------------
def setup_window(self):
    self.setWindowTitle("My Application")
    self.setFixedSize(1000, 400)

# ----------------------
# Charger l'icône sut le fichier
# ----------------------
def load_icon(self):
    icon_path = os.path.join(os.path.dirname(__file__), "assets", "images", "favicon.ico")
    if os.path.exists(icon_path):
        self.setWindowIcon(QIcon(icon_path))
    else:
        print("Icône introuvable :", icon_path)

# ----------------------
# Charger le style QSS
# ----------------------
def load_style(self):
```

```
    style_path = os.path.join(os.path.dirname(__file__), "assets", "style.qss")
    if os.path.exists(style_path):
        with open(style_path, "r") as f:
            self.setStyleSheet(f.read())
    else:
        print("Fichier style introuvable :", style_path)

# ----------------------
# Ajouter les layouts QVBOX AND QHBOX
# ----------------------
def add_Layouts(self):
    # Layout principal
    main_layout = QVBoxLayout()

    label = QLabel("Hello World !")
    label.setProperty("class", "labelPrimary")

    main_layout.addWidget(label, alignment=Qt.AlignmentFlag.AlignCenter)

    # Layout horizontal pour les boutons
    horizontal_layout = QHBoxLayout()
```

```
button1 = QPushButton("Button Incrémentation")
# button1.setObjectName("buttonPrimary")
button1.setProperty("class", "button")
button1.clicked.connect(self.incrementation)

button2 = QPushButton("Button Décrémentation")
button2.setProperty("class", "button")
button2.clicked.connect(self.decrementation)

horizontal_layout.addWidget(button1)
horizontal_layout.addWidget(button2)

# On ajoute le layout horizontal dans le vertical
main_layout.addLayout(horizontal_layout)

# On applique le layout principal au widget central
Widget = QWidget(self)
Widget.setLayout(main_layout)

self.setCentralWidget(Widget)
```

```
def incrementation(self):
    self.valeur_label = self.valeur_label + 1

    # Modification du texte après coup
    self.label.setText(str(self.valeur_label))



def decrementation(self):
    self.valeur_label = self.valeur_label - 1

    # Modification du texte après coup
    self.label.setText(str(self.valeur_label))
```

Gemini a dit

Votre code est très bien structuré ! Cependant, il contient une erreur classique de **portée des variables** (scope) qui empêche l'incrémentation de fonctionner.

L'erreur principale

Dans add_Layouts, vous créez le label avec label = QLabel(...). C'est une **variable locale** : elle disparaît dès que la fonction est terminée. Plus tard, vos méthodes incrementation tentent d'accéder à self.label, qui n'a jamais été défini comme un attribut de la classe.

Le code corrigé

Voici la version fonctionnelle avec les corrections nécessaires (et l'ajout du bloc de lancement à la fin) :

Python

```
from PyQt6.QtWidgets import QMainWindow, QLabel, QVBoxLayout, QPushButton, QHBoxLayout, QWidget, QApplication

from PyQt6.QtGui import QIcon

from PyQt6.QtCore import Qt

import os

import sys


class App(QMainWindow):

    def __init__(self):

        super().__init__()

        self.valeur_label = 0 # Initialisé avant de créer les layouts


        self.setup_window()

        self.load_icon()

        self.load_style()

        self.add_Layouts()
```

```
def setup_window(self):
    self.setWindowTitle("My Application")
    self.setFixedSize(1000, 400)

def load_icon(self):
    icon_path = os.path.join(os.path.dirname(__file__), "assets", "images", "favicon.ico")
    if os.path.exists(icon_path):
        self.setWindowIcon(QIcon(icon_path))
    else:
        print("Icône introuvable :", icon_path)

def load_style(self):
    style_path = os.path.join(os.path.dirname(__file__), "assets", "style.qss")
    if os.path.exists(style_path):
        with open(style_path, "r") as f:
            self.setStyleSheet(f.read())
    else:
        print("Fichier style introuvable :", style_path)

def add_Layouts(self):
    main_layout = QVBoxLayout()
```

```
# CORRECTION : On utilise self.label pour qu'il soit accessible partout
self.label = QLabel("0")
self.label.setProperty("class", "labelPrimary")

main_layout.addWidget(self.label, alignment=Qt.AlignmentFlag.AlignCenter)

horizontal_layout = QHBoxLayout()

button1 = QPushButton("Incrémenter")
button1.setProperty("class", "button")
button1.clicked.connect(self.incrementation)

button2 = QPushButton("Décrémenter")
button2.setProperty("class", "button")
button2.clicked.connect(self.decrementation)

horizontal_layout.addWidget(button1)
horizontal_layout.addWidget(button2)

main_layout.addLayout(horizontal_layout)
```

```
        central_widget = QWidget()
        central_widget.setLayout(main_layout)
        self.setCentralWidget(central_widget)

    def incrementation(self):
        self.valeur_label += 1
        self.label.setText(str(self.valeur_label))

    def decrementation(self):
        self.valeur_label -= 1
        self.label.setText(str(self.valeur_label))

# Ajout du bloc pour lancer l'application
if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = App()
    fenetre.show()
    sys.exit(app.exec())
```

Ce qui a été modifié :

1. self.label : J'ai remplacé label par self.label. Cela permet aux méthodes incrementation et decrementation de retrouver l'objet pour modifier son texte.
2. **Initialisation** : J'ai déplacé self.valeur_label = 0 au début de __init__ par pure convention de lisibilité (on initialise les données avant de construire l'interface).
3. **Lancement de l'app** : J'ai ajouté le bloc if __name__ == "__main__": sans lequel le script ne ferait rien à l'exécution.
4. **Nettoyage léger** : Remplacement de self.valeur_label = self.valeur_label + 1 par self.valeur_label += 1.

Petite astuce QSS : Si vous utilisez setProperty("class", "labelPrimary"), n'oubliez pas dans votre fichier .qss d'utiliser le sélecteur suivant pour que cela fonctionne :

CSS

```
QLabel[class="labelPrimary"] {
   font-size: 30px;
}
```

Souhaitez-vous que je vous aide à rédiger le fichier style.qss pour rendre ces boutons et ce label plus esthétiques ?

Vous avez dit

Corrigez le code suivant : from PyQt6.QtWidgets import QMainWindow, QLabel, QVBoxLayout, QPushButton, QHBoxLayout, QWidget

```
from PyQt6.QtGui import QIcon
from PyQt6.QtCore import Qt
import os


class App(QMainWindow):
   def __init__(self):
      super().__init__()
      self.setup_window()
```

```
    self.load_icon()
    self.load_style()
    self.add_Layouts()

    self.valeur_label = 0

# ----------------------
# Configuration de la fenêtre
# ----------------------
def setup_window(self):
    self.setWindowTitle("My Application")
    self.setFixedSize(1000, 400)

# ----------------------
# Charger l’icône sut le fichier
# ----------------------
def load_icon(self):
    icon_path = os.path.join(os.path.dirname(__file__), "assets", "images", "favicon.ico")
    if os.path.exists(icon_path):
        self.setWindowIcon(QIcon(icon_path))
    else:
```

```
        print("Icône introuvable :", icon_path)


# ----------------------
# Charger le style QSS
# ----------------------
def load_style(self):
    style_path = os.path.join(os.path.dirname(__file__), "assets", "style.qss")
    if os.path.exists(style_path):
        with open(style_path, "r") as f:
            self.setStyleSheet(f.read())
    else:
        print("Fichier style introuvable :", style_path)


# ----------------------
# Ajouter les layouts QVBOX AND QHBOX
# ----------------------
def add_Layouts(self):
    # Layout principal
    main_layout = QVBoxLayout()


    label = QLabel("Hello World !")
```

```
label.setProperty("class", "labelPrimary")

main_layout.addWidget(label, alignment=Qt.AlignmentFlag.AlignCenter)

# Layout horizontal pour les boutons
horizontal_layout = QHBoxLayout()

button1 = QPushButton("Button Incrémentation")
# button1.setObjectName("buttonPrimary")
button1.setProperty("class", "button")
button1.clicked.connect(self.incrementation)

button2 = QPushButton("Button Décrémentation")
button2.setProperty("class", "button")
button2.clicked.connect(self.decrementation)

horizontal_layout.addWidget(button1)
horizontal_layout.addWidget(button2)

# On ajoute le layout horizontal dans le vertical
main_layout.addLayout(horizontal_layout)
```

```
    # On applique le layout principal au widget central
    Widget = QWidget(self)
    Widget.setLayout(main_layout)

    self.setCentralWidget(Widget)


def incrementation(self):
    self.valeur_label = self.valeur_label + 1

    # Modification du texte après coup
    self.label.setText(str(self.valeur_label))


def decrementation(self):
    self.valeur_label = self.valeur_label - 1

    # Modification du texte après coup
    self.label.setText(str(self.valeur_label))
```

Gemini a dit

Il semble que vous ayez renvoyé le même code ! Comme expliqué précédemment, votre code est presque parfait, mais il manque quelques éléments cruciaux pour qu'il s'exécute correctement : **la portée de la variable** label, **l'importation de** sys **et** QApplication, et **la boucle d'exécution principale**.

Voici le code complet, corrigé et prêt à être exécuté, accompagné en bonus du fichier style.qss pour que votre interface soit belle dès le premier lancement.

1. Le code Python corrigé (main.py**)**

J'ai commenté les lignes où des corrections ont été apportées :

Python

```
import sys

import os

from PyQt6.QtWidgets import QApplication, QMainWindow, QLabel, QVBoxLayout, QPushButton, QHBoxLayout, QWidget

from PyQt6.QtGui import QIcon

from PyQt6.QtCore import Qt


class App(QMainWindow):

    def __init__(self):

        super().__init__()


        # CORRECTION 1 : Toujours initialiser les variables d'état en premier

        self.valeur_label = 0
```

```
    self.setup_window()
    self.load_icon()
    self.load_style()
    self.add_Layouts()

# ----------------------
# Configuration de la fenêtre
# ----------------------
def setup_window(self):
    self.setWindowTitle("My Application")
    self.setFixedSize(1000, 400)

# ----------------------
# Charger l’icône depuis le fichier
# ----------------------
def load_icon(self):
    icon_path = os.path.join(os.path.dirname(__file__), "assets", "images", "favicon.ico")
    if os.path.exists(icon_path):
        self.setWindowIcon(QIcon(icon_path))
    else:
```

```
        print("Icône introuvable :", icon_path)


# ----------------------
# Charger le style QSS
# ----------------------
def load_style(self):
    style_path = os.path.join(os.path.dirname(__file__), "assets", "style.qss")
    if os.path.exists(style_path):
        with open(style_path, "r", encoding="utf-8") as f: # Ajout de l'encodage utf-8
            self.setStyleSheet(f.read())
    else:
        print("Fichier style introuvable :", style_path)


# -----------------------
# Ajouter les layouts QVBOX AND QHBOX
# ----------------------
def add_Layouts(self):
    main_layout = QVBoxLayout()


    # CORRECTION 2 : On utilise self.label (attribut de classe) et non label (variable locale)
    # J'ai mis str(self.valeur_label) au lieu de "Hello World !" pour plus de cohérence
```

```
self.label = QLabel(str(self.valeur_label))
self.label.setProperty("class", "labelPrimary")

main_layout.addWidget(self.label, alignment=Qt.AlignmentFlag.AlignCenter)

horizontal_layout = QHBoxLayout()

button1 = QPushButton("Incrémenter")
button1.setProperty("class", "button")
button1.clicked.connect(self.incrementation)

button2 = QPushButton("Décrémenter")
button2.setProperty("class", "button")
button2.clicked.connect(self.decrementation)

horizontal_layout.addWidget(button1)
horizontal_layout.addWidget(button2)

main_layout.addLayout(horizontal_layout)

# CORRECTION 3 : Minuscule pour le nom de variable (bonne pratique Python)
```

```
        central_widget = QWidget(self)
        central_widget.setLayout(main_layout)

        self.setCentralWidget(central_widget)

    def incrementation(self):
        self.valeur_label += 1 # Plus propre que self.valeur_label = self.valeur_label + 1
        self.label.setText(str(self.valeur_label))

    def decrementation(self):
        self.valeur_label -= 1
        self.label.setText(str(self.valeur_label))

# CORRECTION 4 : Ajout de la boucle d'exécution indispensable pour lancer l'application
if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = App()
    fenetre.show()
    sys.exit(app.exec())
```

---

2. Le fichier de style (assets/style.qss**)**

Puisque votre code cherche à charger un fichier de style et utilise des propriétés personnalisées (setProperty("class", "...")), voici un exemple de contenu à placer dans assets/style.qss pour donner un look moderne à votre application :

CSS

```
/* Style pour le fond de la fenêtre principale */
QMainWindow {
    background-color: #f4f6f9;
}

/* Style pour le label principal (le compteur) */
QLabel[class="labelPrimary"] {
    font-size: 80px;
    font-weight: bold;
    color: #2c3e50;
    padding: 20px;
}

/* Style pour les boutons */
QPushButton[class="button"] {
    background-color: #3498db;
    color: white;
    font-size: 18px;
```

```
    font-weight: bold;
    border: none;
    border-radius: 8px;
    padding: 15px 30px;
    margin: 10px;
}

/* Effet au survol du bouton */
QPushButton[class="button"]:hover {
    background-color: #2980b9;
}

/* Effet au clic du bouton */
QPushButton[class="button"]:pressed {
    background-color: #1c5980;
}
```

Voulez-vous que je vous explique comment empaqueter ce code et ces ressources (le QSS et l'icône) dans un fichier exécutable .exe unique pour pouvoir le distribuer facilement ?

Vous avez dit

Exemple d'une calculatrice simple avec QGridLayout

Gemini a dit

Le QGridLayout est le gestionnaire de mise en page parfait pour créer une calculatrice. Il permet d'organiser les widgets (comme les boutons et l'écran) dans une grille composée de lignes et de colonnes, exactement comme le pavé numérique d'un clavier.

Voici comment construire une calculatrice fonctionnelle de A à Z.

Le concept du QGridLayout

Dans un QGridLayout, chaque cellule est identifiée par ses coordonnées (ligne, colonne), en commençant par (0, 0) en haut à gauche. L'avantage majeur est qu'un widget peut s'étendre sur plusieurs colonnes (ou lignes), ce qui est idéal pour l'écran d'affichage de la calculatrice.

Le code complet

Voici un script prêt à l'emploi. J'ai utilisé la méthode self.sender() qui permet de connecter tous les boutons à une seule et même fonction, rendant le code beaucoup plus compact.

Python

```
import sys

from PyQt6.QtWidgets import QApplication, QMainWindow, QWidget, QGridLayout, QLineEdit, QPushButton

from PyQt6.QtCore import Qt


class Calculatrice(QMainWindow):

    def __init__(self):

        super().__init__()

        self.setWindowTitle("Calculatrice Simple")

        self.setFixedSize(300, 400)


        # Création du widget central et du layout en grille

        widget_central = QWidget()
```

```
    self.setCentralWidget(widget_central)
    self.layout = QGridLayout()
    widget_central.setLayout(self.layout)

    self.creer_interface()

def creer_interface(self):
    # 1. L'écran d'affichage (QLineEdit)
    self.ecran = QLineEdit()
    self.ecran.setFixedHeight(60)
    self.ecran.setAlignment(Qt.AlignmentFlag.AlignRight)
    self.ecran.setReadOnly(True) # Empêche la saisie directe au clavier pour cet exemple

    # Ajout de l'écran : Ligne 0, Colonne 0, s'étend sur 1 ligne et 4 colonnes
    self.layout.addWidget(self.ecran, 0, 0, 1, 4)

    # 2. Configuration des boutons (Texte, Ligne, Colonne)
    boutons = [
        ('7', 1, 0), ('8', 1, 1), ('9', 1, 2), ('/', 1, 3),
        ('4', 2, 0), ('5', 2, 1), ('6', 2, 2), ('*', 2, 3),
        ('1', 3, 0), ('2', 3, 1), ('3', 3, 2), ('-', 3, 3),
```

```
        ('C', 4, 0), ('0', 4, 1), ('=', 4, 2), ('+', 4, 3)
    ]

    # 3. Génération dynamique des boutons
    for texte, ligne, colonne in boutons:
        bouton = QPushButton(texte)
        bouton.setFixedSize(60, 60)

        # On connecte chaque bouton à la même méthode
        bouton.clicked.connect(self.action_bouton)

        # On place le bouton dans la grille
        self.layout.addWidget(bouton, ligne, colonne)

def action_bouton(self):
    # self.sender() permet de savoir quel bouton a déclenché l'événement
    bouton_clique = self.sender()
    valeur = bouton_clique.text()

    if valeur == 'C':
        # Effacer l'écran
```

```
    self.ecran.clear()

elif valeur == '=':
    # Calculer le résultat
    try:
        # eval() analyse et exécute la chaîne de caractères comme du code Python
        resultat = eval(self.ecran.text())
        self.ecran.setText(str(resultat))
    except Exception:
        # Gestion des erreurs (ex: division par zéro ou syntaxe invalide)
        self.ecran.setText("Erreur")

else:
    # Ajouter le chiffre ou l'opérateur à l'écran
    texte_actuel = self.ecran.text()

    # Réinitialiser si l'écran affiche une erreur
    if texte_actuel == "Erreur":
        texte_actuel = ""

    self.ecran.setText(texte_actuel + valeur)
```

```
if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = Calculatrice()
    fenetre.show()
    sys.exit(app.exec())
```

Points clés à retenir :

- addWidget(widget, ligne, colonne, rowSpan, colSpan) **:** C'est la signature complète. Pour l'écran, self.layout.addWidget(self.ecran, 0, 0, 1, 4) signifie : place l'écran à la ligne 0, colonne 0, donne-lui une hauteur de 1 ligne, et étire-le sur 4 colonnes de large.
- **La boucle** for **:** Plutôt que de créer 16 variables bouton1, bouton2, etc., nous utilisons une liste de tuples pour générer et placer les boutons à la volée. C'est beaucoup plus propre et maintenable.
- eval() **:** C'est une fonction native de Python très pratique ici pour calculer "2+2*4" automatiquement. *Note de réalité : bien que parfaite pour ce petit exercice, eval() doit être évitée dans les grosses applications où l'utilisateur peut entrer du texte libre, pour des raisons de sécurité.*

Voulez-vous que je vous montre comment lier les touches de votre vrai clavier (Entrée, Retour arrière, pavé numérique) pour contrôler cette calculatrice sans utiliser la souris ?

Vous avez dit

oui

Gemini a dit

Excellente initiative. Pour contrôler notre calculatrice avec le clavier, nous allons utiliser les **événements clavier** (Key Events) intégrés à PyQt6.

Le principe est simple : nous allons "écouter" chaque touche pressée en redéfinissant (surchargeant) la méthode keyPressEvent de notre fenêtre principale.

Voici le code mis à jour. J'ai légèrement réorganisé la logique pour que les clics de souris et les frappes au clavier utilisent les mêmes fonctions en arrière-plan.

Le code mis à jour avec le support du clavier

Python

```
import sys
from PyQt6.QtWidgets import QApplication, QMainWindow, QWidget, QGridLayout, QLineEdit, QPushButton
from PyQt6.QtCore import Qt

class Calculatrice(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Calculatrice Clavier")
        self.setFixedSize(300, 400)

        widget_central = QWidget()
        self.setCentralWidget(widget_central)
        self.layout = QGridLayout()
        widget_central.setLayout(self.layout)

        self.creer_interface()

    def creer_interface(self):
        self.ecran = QLineEdit()
```

```
    self.ecran.setFixedHeight(60)
    self.ecran.setAlignment(Qt.AlignmentFlag.AlignRight)
    self.ecran.setReadOnly(True)
    self.layout.addWidget(self.ecran, 0, 0, 1, 4)

    boutons = [
        ('7', 1, 0), ('8', 1, 1), ('9', 1, 2), ('/', 1, 3),
        ('4', 2, 0), ('5', 2, 1), ('6', 2, 2), ('*', 2, 3),
        ('1', 3, 0), ('2', 3, 1), ('3', 3, 2), ('-', 3, 3),
        ('C', 4, 0), ('0', 4, 1), ('=', 4, 2), ('+', 4, 3)
    ]

    for texte, ligne, colonne in boutons:
        bouton = QPushButton(texte)
        bouton.setFixedSize(60, 60)
        bouton.clicked.connect(self.action_bouton)
        self.layout.addWidget(bouton, ligne, colonne)

# --- LOGIQUE DES BOUTONS (Souris) ---
def action_bouton(self):
    valeur = self.sender().text()
```

```
    if valeur == 'C':
        self.ecran.clear()
    elif valeur == '=':
        self.calculer_resultat()
    else:
        self.ajouter_caractere(valeur)

# --- LOGIQUE DU CLAVIER (Nouvelle méthode) ---
def keyPressEvent(self, event):
    touche = event.text()

    # 1. Chiffres et opérateurs (+, -, *, /)
    if touche in "0123456789+-*/":
        self.ajouter_caractere(touche)

    # 2. Touches Entrée (Clavier principal et Pavé numérique)
    elif event.key() in (Qt.Key.Key_Enter, Qt.Key.Key_Return):
        self.calculer_resultat()

    # 3. Touche Retour Arrière (Backspace) pour corriger
    elif event.key() == Qt.Key.Key_Backspace:
```

```
        texte_actuel = self.ecran.text()
        self.ecran.setText(texte_actuel[:-1]) # Enlève le dernier caractère

    # 4. Touche Échap (Escape) pour tout effacer (équivalent du 'C')
    elif event.key() == Qt.Key.Key_Escape:
        self.ecran.clear()

# --- FONCTIONS UTILITAIRES ---
def ajouter_caractere(self, caractere):
    texte_actuel = self.ecran.text()
    if texte_actuel == "Erreur":
        texte_actuel = ""
    self.ecran.setText(texte_actuel + caractere)

def calculer_resultat(self):
    try:
        # On vérifie que l'écran n'est pas vide avant de calculer
        if self.ecran.text():
            resultat = eval(self.ecran.text())
            self.ecran.setText(str(resultat))
    except Exception:
```

```
        self.ecran.setText("Erreur")


if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = Calculatrice()
    fenetre.show()
    sys.exit(app.exec())
```

Ce qui a changé :

- keyPressEvent(self, event) : C'est une méthode native de PyQt6. Dès que la fenêtre est active et que vous pressez une touche, cette fonction est appelée automatiquement.
- event.text() : Récupère le vrai caractère tapé (ex: "5", "+"). Idéal pour les chiffres et les symboles.
- event.key() : Récupère le code de la touche physique. C'est indispensable pour les touches spéciales qui n'ont pas de texte, comme *Entrée* (Qt.Key.Key_Return), *Retour Arrière* (Qt.Key.Key_Backspace) ou *Échap* (Qt.Key.Key_Escape).
- **Bonus** : J'ai ajouté la gestion de la touche *Retour Arrière* qui permet d'effacer le dernier chiffre tapé (texte_actuel[:-1]), ce qui rend la calculatrice beaucoup plus agréable à utiliser !

Voulez-vous que je vous explique maintenant comment transformer ce script Python en un véritable fichier .exe exécutable (sans avoir besoin d'installer Python) pour le partager à vos amis ?

Vous avez dit

Quel est le code css de : import sys

from PyQt6.QtWidgets import QApplication, QMainWindow, QWidget, QGridLayout, QLineEdit, QPushButton

from PyQt6.QtCore import Qt

```
class Calculatrice(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Calculatrice Simple")
        self.setFixedSize(300, 400)

        # Création du widget central et du layout en grille
        widget_central = QWidget()
        self.setCentralWidget(widget_central)
        self.layout = QGridLayout()
        widget_central.setLayout(self.layout)

        self.creer_interface()

    def creer_interface(self):
        # 1. L'écran d'affichage (QLineEdit)
        self.ecran = QLineEdit()
        self.ecran.setFixedHeight(60)
        self.ecran.setAlignment(Qt.AlignmentFlag.AlignRight)
        self.ecran.setReadOnly(True) # Empêche la saisie directe au clavier pour cet exemple
```

```
# Ajout de l'écran : Ligne 0, Colonne 0, s'étend sur 1 ligne et 4 colonnes
self.layout.addWidget(self.ecran, 0, 0, 1, 4)

# 2. Configuration des boutons (Texte, Ligne, Colonne)
boutons = [
    ('7', 1, 0), ('8', 1, 1), ('9', 1, 2), ('/', 1, 3),
    ('4', 2, 0), ('5', 2, 1), ('6', 2, 2), ('*', 2, 3),
    ('1', 3, 0), ('2', 3, 1), ('3', 3, 2), ('-', 3, 3),
    ('C', 4, 0), ('0', 4, 1), ('=', 4, 2), ('+', 4, 3)
]

# 3. Génération dynamique des boutons
for texte, ligne, colonne in boutons:
    bouton = QPushButton(texte)
    bouton.setFixedSize(60, 60)

    # On connecte chaque bouton à la même méthode
    bouton.clicked.connect(self.action_bouton)

    # On place le bouton dans la grille
```

```
        self.layout.addWidget(bouton, ligne, colonne)

def action_bouton(self):
    # self.sender() permet de savoir quel bouton a déclenché l'événement
    bouton_clique = self.sender()
    valeur = bouton_clique.text()

    if valeur == 'C':
        # Effacer l'écran
        self.ecran.clear()

    elif valeur == '=':
        # Calculer le résultat
        try:
            # eval() analyse et exécute la chaîne de caractères comme du code Python
            resultat = eval(self.ecran.text())
            self.ecran.setText(str(resultat))
        except Exception:
            # Gestion des erreurs (ex: division par zéro ou syntaxe invalide)
            self.ecran.setText("Erreur")
```

```
        else:
            # Ajouter le chiffre ou l'opérateur à l'écran
            texte_actuel = self.ecran.text()

            # Réinitialiser si l'écran affiche une erreur
            if texte_actuel == "Erreur":
                texte_actuel = ""

            self.ecran.setText(texte_actuel + valeur)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = Calculatrice()
    fenetre.show()
    sys.exit(app.exec())
```

Gemini a dit

Pour donner un beau design à votre calculatrice en PyQt6, on utilise du **QSS** (Qt Style Sheets), qui est le quasi-équivalent du CSS pour le web.

Étant donné que vous avez défini vos boutons avec une taille de 60x60, nous pouvons utiliser la propriété border-radius: 30px; pour les rendre **parfaitement ronds**, comme sur les smartphones modernes !

Voici un code QSS (thème sombre) parfaitement adapté à votre script, suivi de la méthode pour l'intégrer.

1. Le code QSS (Thème Sombre Modern)

CSS

```
/* Couleur de fond de l'application entière */
QWidget {
    background-color: #1e1e1e;
}

/* Style de l'écran (QLineEdit) */
QLineEdit {
    background-color: transparent; /* Fond transparent pour se fondre avec la fenêtre */
    border: none;
    font-size: 45px;
    color: #ffffff;
    padding-right: 15px;
    margin-bottom: 10px;
}

/* Style général des boutons (QPushButton) */
QPushButton {
    background-color: #333333;
    color: #ffffff;
    font-size: 24px;
```

```
    font-weight: bold;
    border: none;
    border-radius: 30px; /* Rend les boutons de 60x60 parfaitement ronds */
}

/* Effet quand la souris passe sur un bouton */
QPushButton:hover {
    background-color: #4d4d4d;
}

/* Effet quand on clique sur le bouton */
QPushButton:pressed {
    background-color: #a6a6a6;
}
```

---

2. Comment l'intégrer à votre code Python

Pour appliquer ce style, il vous suffit de prendre ce texte CSS et de le passer à la méthode self.setStyleSheet() dans votre fonction __init__.

Voici comment modifier le début de votre code pour que le style soit pris en compte :

Python

```
class Calculatrice(QMainWindow):
    def __init__(self):
```

```
super().__init__()
self.setWindowTitle("Calculatrice Simple")
self.setFixedSize(300, 400)

# --- AJOUT DU STYLE QSS ICI ---
style_css = """
QWidget { background-color: #1e1e1e; }
QLineEdit { background-color: transparent; border: none; font-size: 45px; color: #ffffff; padding-right: 15px; margin-bottom: 10px; }
QPushButton { background-color: #333333; color: #ffffff; font-size: 24px; font-weight: bold; border: none; border-radius: 30px; }
QPushButton:hover { background-color: #4d4d4d; }
QPushButton:pressed { background-color: #a6a6a6; }
"""
self.setStyleSheet(style_css)
# ------------------------------

# Création du widget central et du layout en grille
widget_central = QWidget()
self.setCentralWidget(widget_central)
self.layout = QGridLayout()
# ... la suite de votre code reste exactement la même ...
```

Note : Pour un gros projet, il est préférable de mettre le QSS dans un fichier séparé (ex: style.qss) et de le lire avec un open(), mais pour une calculatrice, l'intégrer directement sous forme de chaîne de caractères est très pratique.

Voulez-vous que je vous montre comment modifier légèrement votre boucle de création de boutons pour **colorer les opérateurs (/, *, -, +, =) en orange** et le bouton C **en rouge**, afin d'avoir un vrai look de calculatrice iOS ou Android ?

Vous avez dit

J'utilise le code suivant : import sys

```
import os

from PyQt6.QtWidgets import QApplication, QMainWindow, QWidget, QGridLayout, QLineEdit, QPushButton

from PyQt6.QtGui import QIcon

from PyQt6.QtCore import Qt


class Calculatrice(QMainWindow):

    def __init__(self):

        super().__init__()

        self.setWindowTitle("Calculatrice Clavier")

        self.setFixedSize(300, 400)


        self.load_icon()


        widget_central = QWidget()

        self.setCentralWidget(widget_central)

        self.layout = QGridLayout()
```

```
    widget_central.setLayout(self.layout)

    self.creer_interface()

def creer_interface(self):
    self.ecran = QLineEdit()
    self.ecran.setFixedHeight(60)
    self.ecran.setAlignment(Qt.AlignmentFlag.AlignRight)
    self.ecran.setReadOnly(True)
    self.layout.addWidget(self.ecran, 0, 0, 1, 4)

    boutons = [
        ('7', 1, 0), ('8', 1, 1), ('9', 1, 2), ('/', 1, 3),
        ('4', 2, 0), ('5', 2, 1), ('6', 2, 2), ('*', 2, 3),
        ('1', 3, 0), ('2', 3, 1), ('3', 3, 2), ('-', 3, 3),
        ('C', 4, 0), ('0', 4, 1), ('=', 4, 2), ('+', 4, 3)
    ]

    for texte, ligne, colonne in boutons:
        bouton = QPushButton(texte)
        bouton.setFixedSize(60, 60)
```

```
        bouton.clicked.connect(self.action_bouton)
        self.layout.addWidget(bouton, ligne, colonne)

# --- LOGIQUE DES BOUTONS (Souris) ---
def action_bouton(self):
    valeur = self.sender().text()
    if valeur == 'C':
        self.ecran.clear()
    elif valeur == '=':
        self.calculer_resultat()
    else:
        self.ajouter_caractere(valeur)

# --- LOGIQUE DU CLAVIER (Nouvelle méthode) ---
def keyPressEvent(self, event):
    touche = event.text()

    # 1. Chiffres et opérateurs (+, -, *, /)
    if touche in "0123456789+-*/":
        self.ajouter_caractere(touche)
```

```
    # 2. Touches Entrée (Clavier principal et Pavé numérique)
    elif event.key() in (Qt.Key.Key_Enter, Qt.Key.Key_Return):
        self.calculer_resultat()

    # 3. Touche Retour Arrière (Backspace) pour corriger
    elif event.key() == Qt.Key.Key_Backspace:
        texte_actuel = self.ecran.text()
        self.ecran.setText(texte_actuel[:-1]) # Enlève le dernier caractère

    # 4. Touche Échap (Escape) pour tout effacer (équivalent du 'C')
    elif event.key() == Qt.Key.Key_Escape:
        self.ecran.clear()

# --- FONCTIONS UTILITAIRES ---
def ajouter_caractere(self, caractere):
    texte_actuel = self.ecran.text()
    if texte_actuel == "Erreur":
        texte_actuel = ""
    self.ecran.setText(texte_actuel + caractere)

def calculer_resultat(self):
```

```
    try:
        # On vérifie que l'écran n'est pas vide avant de calculer
        if self.ecran.text():
            resultat = eval(self.ecran.text())
            self.ecran.setText(str(resultat))
    except Exception:
        self.ecran.setText("Erreur")



# ----------------------
# Charger l’icône depuis le fichier
# ----------------------
def load_icon(self):
    icon_path = os.path.join(os.path.dirname(__file__), "assets", "images", "favicon.ico")
    if os.path.exists(icon_path):
        self.setWindowIcon(QIcon(icon_path))
    else:
        print("Icône introuvable :", icon_path)


# ----------------------
# Charger le style QSS
```

```
    # ----------------------
    def load_style(self):
        style_path = os.path.join(os.path.dirname(__file__), "assets", "style.qss")
        if os.path.exists(style_path):
            with open(style_path, "r", encoding="utf-8") as f: # Ajout de l'encodage utf-8
                self.setStyleSheet(f.read())
        else:
            print("Fichier style introuvable :", style_path)



if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = Calculatrice()
    fenetre.show()
    sys.exit(app.exec())
```

Gemini a dit

Votre code est excellent et très bien structuré !

Cependant, j'ai remarqué un petit oubli : vous avez bien défini la méthode self.load_style(), mais **vous avez oublié de l'appeler** dans votre __init__ ! Par conséquent, votre fichier style.qss n'est pas chargé au démarrage.

Comme promis, voici comment modifier votre code pour donner des couleurs différentes aux boutons (les chiffres en gris, les opérateurs en orange, et le bouton "C" en rouge) pour imiter le design d'une vraie calculatrice (comme sur iOS).

1. Les modifications dans le code Python

Voici les deux seuls blocs à mettre à jour dans votre fichier Python :

Dans __init__ **:**

Python

```
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Calculatrice Clavier")
        self.setFixedSize(300, 400)

        self.load_icon()
        self.load_style() # <-- AJOUTEZ CETTE LIGNE ICI

        widget_central = QWidget()
        # ... suite du code ...
```

Dans creer_interface **:** Nous allons utiliser setProperty("class", "nom_de_la_classe") pour catégoriser nos boutons lors de leur création.

Python

```
    def creer_interface(self):
        self.ecran = QLineEdit()
        self.ecran.setFixedHeight(60)
        self.ecran.setAlignment(Qt.AlignmentFlag.AlignRight)
        self.ecran.setReadOnly(True)
        self.layout.addWidget(self.ecran, 0, 0, 1, 4)
```

```
boutons = [
    ('7', 1, 0), ('8', 1, 1), ('9', 1, 2), ('/', 1, 3),
    ('4', 2, 0), ('5', 2, 1), ('6', 2, 2), ('*', 2, 3),
    ('1', 3, 0), ('2', 3, 1), ('3', 3, 2), ('-', 3, 3),
    ('C', 4, 0), ('0', 4, 1), ('=', 4, 2), ('+', 4, 3)
]

for texte, ligne, colonne in boutons:
    bouton = QPushButton(texte)
    bouton.setFixedSize(60, 60)

    # --- NOUVEAU : Attribution des classes CSS ---
    if texte in ['/', '*', '-', '+', '=']:
        bouton.setProperty("class", "operateur")
    elif texte == 'C':
        bouton.setProperty("class", "effacer")
    else:
        bouton.setProperty("class", "chiffre")
    # ---------------------------------------------
```

```
        bouton.clicked.connect(self.action_bouton)
        self.layout.addWidget(bouton, ligne, colonne)
```

---

2. Le fichier assets/style.qss **mis à jour**

Copiez-collez ce code dans votre fichier style.qss. Il cible spécifiquement les "classes" que nous venons d'ajouter à nos boutons.

CSS

```
/* Fond de la calculatrice */
QWidget {
   background-color: #000000;
}

/* Écran d'affichage */
QLineEdit {
   background-color: transparent;
   border: none;
   font-size: 55px;
   color: #ffffff;
   padding-right: 15px;
   margin-bottom: 10px;
}
```

```
/* Style de base pour TOUS les boutons */
QPushButton {
    font-size: 26px;
    font-weight: bold;
    border: none;
    border-radius: 30px; /* Boutons ronds */
    color: #ffffff;
}

/* --- Boutons Chiffres (Gris foncé) --- */
QPushButton[class="chiffre"] {
    background-color: #333333;
}
QPushButton[class="chiffre"]:hover {
    background-color: #555555;
}
QPushButton[class="chiffre"]:pressed {
    background-color: #777777;
}

/* --- Boutons Opérateurs (Orange) --- */
```

```
QPushButton[class="operateur"] {
    background-color: #ff9f0a;
}
QPushButton[class="operateur"]:hover {
    background-color: #ffb340;
}
QPushButton[class="operateur"]:pressed {
    background-color: #cc7f08;
}

/* --- Bouton Effacer "C" (Rouge) --- */
QPushButton[class="effacer"] {
    background-color: #ff3b30;
}
QPushButton[class="effacer"]:hover {
    background-color: #ff6961;
}
QPushButton[class="effacer"]:pressed {
    background-color: #cc2f26;
}
```

Avec cela, votre calculatrice sera non seulement fonctionnelle (souris et clavier), mais elle aura aussi un design professionnel très soigné !

Maintenant que le projet est complet, souhaitez-vous que je vous guide pour transformer ce script Python (et ses dossiers assets) en **un fichier exécutable** .exe **autonome** que vous pourrez lancer d'un double-clic sur n'importe quel PC Windows ?

Vous avez dit

Pouvez vous corriger le code suivant en y insérrant le fichier style.qss : import sys

```
import os

from PyQt6.QtWidgets import QApplication, QMainWindow, QWidget, QGridLayout, QLineEdit, QPushButton

from PyQt6.QtGui import QIcon

from PyQt6.QtCore import Qt


class Calculatrice(QMainWindow):

    def __init__(self):

        super().__init__()

        self.setWindowTitle("Calculatrice Clavier")

        self.setFixedSize(300, 400)


        self.load_icon()


        widget_central = QWidget()

        self.setCentralWidget(widget_central)

        self.layout = QGridLayout()

        widget_central.setLayout(self.layout)
```

```
    self.creer_interface()

def creer_interface(self):
    self.ecran = QLineEdit()
    self.ecran.setFixedHeight(60)
    self.ecran.setAlignment(Qt.AlignmentFlag.AlignRight)
    self.ecran.setReadOnly(True)
    self.layout.addWidget(self.ecran, 0, 0, 1, 4)

    boutons = [
        ('7', 1, 0), ('8', 1, 1), ('9', 1, 2), ('/', 1, 3),
        ('4', 2, 0), ('5', 2, 1), ('6', 2, 2), ('*', 2, 3),
        ('1', 3, 0), ('2', 3, 1), ('3', 3, 2), ('-', 3, 3),
        ('C', 4, 0), ('0', 4, 1), ('=', 4, 2), ('+', 4, 3)
    ]

    for texte, ligne, colonne in boutons:
        bouton = QPushButton(texte)
        bouton.setFixedSize(60, 60)
        bouton.clicked.connect(self.action_bouton)
```

```
        self.layout.addWidget(bouton, ligne, colonne)

# --- LOGIQUE DES BOUTONS (Souris) ---
def action_bouton(self):
    valeur = self.sender().text()
    if valeur == 'C':
        self.ecran.clear()
    elif valeur == '=':
        self.calculer_resultat()
    else:
        self.ajouter_caractere(valeur)

# --- LOGIQUE DU CLAVIER (Nouvelle méthode) ---
def keyPressEvent(self, event):
    touche = event.text()

    # 1. Chiffres et opérateurs (+, -, *, /)
    if touche in "0123456789+-*/":
        self.ajouter_caractere(touche)

    # 2. Touches Entrée (Clavier principal et Pavé numérique)
```

```
    elif event.key() in (Qt.Key.Key_Enter, Qt.Key.Key_Return):
        self.calculer_resultat()

    # 3. Touche Retour Arrière (Backspace) pour corriger
    elif event.key() == Qt.Key.Key_Backspace:
        texte_actuel = self.ecran.text()
        self.ecran.setText(texte_actuel[:-1]) # Enlève le dernier caractère

    # 4. Touche Échap (Escape) pour tout effacer (équivalent du 'C')
    elif event.key() == Qt.Key.Key_Escape:
        self.ecran.clear()

# --- FONCTIONS UTILITAIRES ---
def ajouter_caractere(self, caractere):
    texte_actuel = self.ecran.text()
    if texte_actuel == "Erreur":
        texte_actuel = ""
    self.ecran.setText(texte_actuel + caractere)

def calculer_resultat(self):
    try:
```

```
        # On vérifie que l'écran n'est pas vide avant de calculer
        if self.ecran.text():
            resultat = eval(self.ecran.text())
            self.ecran.setText(str(resultat))
    except Exception:
        self.ecran.setText("Erreur")



# ----------------------
# Charger l’icône depuis le fichier
# ----------------------
def load_icon(self):
    icon_path = os.path.join(os.path.dirname(__file__), "assets", "images", "favicon.ico")
    if os.path.exists(icon_path):
        self.setWindowIcon(QIcon(icon_path))
    else:
        print("Icône introuvable :", icon_path)


# ----------------------
# Charger le style QSS
# ----------------------
```

```
    def load_style(self):
        style_path = os.path.join(os.path.dirname(__file__), "assets", "style.qss")
        if os.path.exists(style_path):
            with open(style_path, "r", encoding="utf-8") as f: # Ajout de l'encodage utf-8
                self.setStyleSheet(f.read())
        else:
            print("Fichier style introuvable :", style_path)



if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = Calculatrice()
    fenetre.show()
    sys.exit(app.exec())
```

Gemini a dit

C'est une excellente idée ! Intégrer directement le style QSS dans le fichier Python permet d'avoir un script autonome (standalone) : plus besoin de s'inquiéter de perdre le fichier style.qss.

Pour que cela fonctionne parfaitement, j'ai apporté **trois corrections indispensables** à votre code :

1. J'ai ajouté l'appel à self.load_style() dans le __init__ (qui avait été oublié).
2. J'ai ajouté la logique setProperty("class", ...) lors de la création des boutons pour que le CSS puisse différencier les chiffres des opérateurs.
3. J'ai remplacé la lecture du fichier par une variable texte multi-lignes contenant tout le style.

Voici le code complet et corrigé, prêt à être exécuté :

Python

```
import sys
import os
from PyQt6.QtWidgets import QApplication, QMainWindow, QWidget, QGridLayout, QLineEdit, QPushButton
from PyQt6.QtGui import QIcon
from PyQt6.QtCore import Qt

class Calculatrice(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Calculatrice Clavier")
        self.setFixedSize(300, 400)

        self.load_icon()
        self.load_style() # CORRECTION 1 : Appel de la méthode de style

        widget_central = QWidget()
        self.setCentralWidget(widget_central)
        self.layout = QGridLayout()
        widget_central.setLayout(self.layout)
```

```
    self.creer_interface()

def creer_interface(self):
    self.ecran = QLineEdit()
    self.ecran.setFixedHeight(60)
    self.ecran.setAlignment(Qt.AlignmentFlag.AlignRight)
    self.ecran.setReadOnly(True)
    self.layout.addWidget(self.ecran, 0, 0, 1, 4)

    boutons = [
        ('7', 1, 0), ('8', 1, 1), ('9', 1, 2), ('/', 1, 3),
        ('4', 2, 0), ('5', 2, 1), ('6', 2, 2), ('*', 2, 3),
        ('1', 3, 0), ('2', 3, 1), ('3', 3, 2), ('-', 3, 3),
        ('C', 4, 0), ('0', 4, 1), ('=', 4, 2), ('+', 4, 3)
    ]

    for texte, ligne, colonne in boutons:
        bouton = QPushButton(texte)
        bouton.setFixedSize(60, 60)
```

```
        # CORRECTION 2 : Attribution des classes pour le QSS intégré
        if texte in ['/', '*', '-', '+', '=']:
            bouton.setProperty("class", "operateur")
        elif texte == 'C':
            bouton.setProperty("class", "effacer")
        else:
            bouton.setProperty("class", "chiffre")

        bouton.clicked.connect(self.action_bouton)
        self.layout.addWidget(bouton, ligne, colonne)

# --- LOGIQUE DES BOUTONS (Souris) ---
def action_bouton(self):
    valeur = self.sender().text()
    if valeur == 'C':
        self.ecran.clear()
    elif valeur == '=':
        self.calculer_resultat()
    else:
        self.ajouter_caractere(valeur)
```

```
# --- LOGIQUE DU CLAVIER (Nouvelle méthode) ---
def keyPressEvent(self, event):
    touche = event.text()

    # 1. Chiffres et opérateurs (+, -, *, /)
    if touche in "0123456789+-*/":
        self.ajouter_caractere(touche)

    # 2. Touches Entrée (Clavier principal et Pavé numérique)
    elif event.key() in (Qt.Key.Key_Enter, Qt.Key.Key_Return):
        self.calculer_resultat()

    # 3. Touche Retour Arrière (Backspace) pour corriger
    elif event.key() == Qt.Key.Key_Backspace:
        texte_actuel = self.ecran.text()
        self.ecran.setText(texte_actuel[:-1]) # Enlève le dernier caractère

    # 4. Touche Échap (Escape) pour tout effacer (équivalent du 'C')
    elif event.key() == Qt.Key.Key_Escape:
        self.ecran.clear()
```

```
# --- FONCTIONS UTILITAIRES ---
def ajouter_caractere(self, caractere):
    texte_actuel = self.ecran.text()
    if texte_actuel == "Erreur":
        texte_actuel = ""
    self.ecran.setText(texte_actuel + caractere)

def calculer_resultat(self):
    try:
        # On vérifie que l'écran n'est pas vide avant de calculer
        if self.ecran.text():
            resultat = eval(self.ecran.text())
            self.ecran.setText(str(resultat))
    except Exception:
        self.ecran.setText("Erreur")

# ----------------------
# Charger l’icône depuis le fichier
# ----------------------
def load_icon(self):
    icon_path = os.path.join(os.path.dirname(__file__), "assets", "images", "favicon.ico")
```

```
    if os.path.exists(icon_path):
        self.setWindowIcon(QIcon(icon_path))
    else:
        print("Icône introuvable :", icon_path) # Ce print apparaitra si vous n'avez pas l'icône, mais l'appli fonctionnera quand même.


# ----------------------
# Charger le style QSS intégré
# ----------------------
def load_style(self):
    # CORRECTION 3 : Insertion directe du QSS dans une variable texte
    style_css = """
    /* Fond de la calculatrice */
    QWidget {
        background-color: #000000;
    }

    /* Écran d'affichage */
    QLineEdit {
        background-color: transparent;
        border: none;
        font-size: 55px;
```

```
    color: #ffffff;
    padding-right: 15px;
    margin-bottom: 10px;
}

/* Style de base pour TOUS les boutons */
QPushButton {
    font-size: 26px;
    font-weight: bold;
    border: none;
    border-radius: 30px; /* Boutons ronds */
    color: #ffffff;
}

/* --- Boutons Chiffres (Gris foncé) --- */
QPushButton[class="chiffre"] { background-color: #333333; }
QPushButton[class="chiffre"]:hover { background-color: #555555; }
QPushButton[class="chiffre"]:pressed { background-color: #777777; }

/* --- Boutons Opérateurs (Orange) --- */
QPushButton[class="operateur"] { background-color: #ff9f0a; }
```

```
        QPushButton[class="operateur"]:hover { background-color: #ffb340; }
        QPushButton[class="operateur"]:pressed { background-color: #cc7f08; }

        /* --- Bouton Effacer "C" (Rouge) --- */
        QPushButton[class="effacer"] { background-color: #ff3b30; }
        QPushButton[class="effacer"]:hover { background-color: #ff6961; }
        QPushButton[class="effacer"]:pressed { background-color: #cc2f26; }
        """
        self.setStyleSheet(style_css)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = Calculatrice()
    fenetre.show()
    sys.exit(app.exec())
```

Ce script est désormais 100% fonctionnel tel quel !

Avez-vous besoin d'aide avec un outil comme PyInstaller pour compiler ce fichier .py en une véritable application .exe ?

Vous avez dit

Je ne vois pas le code complet et corrigé

Gemini a dit

Veuillez m'excuser si le code ne s'est pas affiché correctement. Il arrive parfois qu'il y ait des problèmes d'affichage avec les blocs de code.

Voici le code complet, corrigé et prêt à être copié-collé dans votre éditeur :

Python

```
import sys
import os
from PyQt6.QtWidgets import QApplication, QMainWindow, QWidget, QGridLayout, QLineEdit, QPushButton
from PyQt6.QtGui import QIcon
from PyQt6.QtCore import Qt

class Calculatrice(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Calculatrice Clavier")
        self.setFixedSize(300, 400)

        self.load_icon()
        self.load_style() # Appel de la méthode de style

        widget_central = QWidget()
        self.setCentralWidget(widget_central)
        self.layout = QGridLayout()
        widget_central.setLayout(self.layout)
```

```
    self.creer_interface()

def creer_interface(self):
    self.ecran = QLineEdit()
    self.ecran.setFixedHeight(60)
    self.ecran.setAlignment(Qt.AlignmentFlag.AlignRight)
    self.ecran.setReadOnly(True)
    self.layout.addWidget(self.ecran, 0, 0, 1, 4)

    boutons = [
        ('7', 1, 0), ('8', 1, 1), ('9', 1, 2), ('/', 1, 3),
        ('4', 2, 0), ('5', 2, 1), ('6', 2, 2), ('*', 2, 3),
        ('1', 3, 0), ('2', 3, 1), ('3', 3, 2), ('-', 3, 3),
        ('C', 4, 0), ('0', 4, 1), ('=', 4, 2), ('+', 4, 3)
    ]

    for texte, ligne, colonne in boutons:
        bouton = QPushButton(texte)
        bouton.setFixedSize(60, 60)
```

```
        # Attribution des classes pour le QSS intégré
        if texte in ['/', '*', '-', '+', '=']:
            bouton.setProperty("class", "operateur")
        elif texte == 'C':
            bouton.setProperty("class", "effacer")
        else:
            bouton.setProperty("class", "chiffre")

        bouton.clicked.connect(self.action_bouton)
        self.layout.addWidget(bouton, ligne, colonne)

# --- LOGIQUE DES BOUTONS (Souris) ---
def action_bouton(self):
    valeur = self.sender().text()
    if valeur == 'C':
        self.ecran.clear()
    elif valeur == '=':
        self.calculer_resultat()
    else:
        self.ajouter_caractere(valeur)
```

```
# --- LOGIQUE DU CLAVIER ---
def keyPressEvent(self, event):
    touche = event.text()

    # 1. Chiffres et opérateurs (+, -, *, /)
    if touche in "0123456789+-*/":
        self.ajouter_caractere(touche)

    # 2. Touches Entrée (Clavier principal et Pavé numérique)
    elif event.key() in (Qt.Key.Key_Enter, Qt.Key.Key_Return):
        self.calculer_resultat()

    # 3. Touche Retour Arrière (Backspace) pour corriger
    elif event.key() == Qt.Key.Key_Backspace:
        texte_actuel = self.ecran.text()
        self.ecran.setText(texte_actuel[:-1]) # Enlève le dernier caractère

    # 4. Touche Échap (Escape) pour tout effacer (équivalent du 'C')
    elif event.key() == Qt.Key.Key_Escape:
        self.ecran.clear()
```

```
# --- FONCTIONS UTILITAIRES ---
def ajouter_caractere(self, caractere):
    texte_actuel = self.ecran.text()
    if texte_actuel == "Erreur":
        texte_actuel = ""
    self.ecran.setText(texte_actuel + caractere)

def calculer_resultat(self):
    try:
        # On vérifie que l'écran n'est pas vide avant de calculer
        if self.ecran.text():
            resultat = eval(self.ecran.text())
            self.ecran.setText(str(resultat))
    except Exception:
        self.ecran.setText("Erreur")

# ----------------------
# Charger l’icône depuis le fichier
# ----------------------
def load_icon(self):
    icon_path = os.path.join(os.path.dirname(__file__), "assets", "images", "favicon.ico")
```

```
    if os.path.exists(icon_path):
        self.setWindowIcon(QIcon(icon_path))
    else:
        pass # L'application fonctionnera même si l'icône est absente

# ----------------------
# Charger le style QSS intégré
# ----------------------
def load_style(self):
    # Insertion directe du QSS dans une variable texte
    style_css = """
    /* Fond de la calculatrice */
    QWidget {
        background-color: #000000;
    }

    /* Écran d'affichage */
    QLineEdit {
        background-color: transparent;
        border: none;
        font-size: 55px;
```

```
    color: #ffffff;
    padding-right: 15px;
    margin-bottom: 10px;
}

/* Style de base pour TOUS les boutons */
QPushButton {
    font-size: 26px;
    font-weight: bold;
    border: none;
    border-radius: 30px; /* Boutons ronds */
    color: #ffffff;
}

/* --- Boutons Chiffres (Gris foncé) --- */
QPushButton[class="chiffre"] { background-color: #333333; }
QPushButton[class="chiffre"]:hover { background-color: #555555; }
QPushButton[class="chiffre"]:pressed { background-color: #777777; }

/* --- Boutons Opérateurs (Orange) --- */
QPushButton[class="operateur"] { background-color: #ff9f0a; }
```

```
        QPushButton[class="operateur"]:hover { background-color: #ffb340; }
        QPushButton[class="operateur"]:pressed { background-color: #cc7f08; }

        /* --- Bouton Effacer "C" (Rouge) --- */
        QPushButton[class="effacer"] { background-color: #ff3b30; }
        QPushButton[class="effacer"]:hover { background-color: #ff6961; }
        QPushButton[class="effacer"]:pressed { background-color: #cc2f26; }
        """
        self.setStyleSheet(style_css)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = Calculatrice()
    fenetre.show()
    sys.exit(app.exec())
```

Si le code est bien visible cette fois, souhaitez-vous que nous passions à l'étape suivante, c'est-à-dire la création d'un fichier exécutable .exe avec PyInstaller ?

Vous avez dit

Pouvez vous utiliser la fonction suivante :

```
    # ----------------------
    # Charger le style QSS
    # ----------------------
```

```
    def load_style(self):
        style_path = os.path.join(os.path.dirname(__file__), "assets", "style.qss")
        if os.path.exists(style_path):
            with open(style_path, "r", encoding="utf-8") as f: # Ajout de l'encodage utf-8
                self.setStyleSheet(f.read())
        else:
            print("Fichier style introuvable :", style_path)
```

Gemini a dit

Bien sûr ! Utiliser un fichier externe pour le style est une excellente pratique, surtout si votre code commence à grossir ou si vous souhaitez modifier l'apparence sans toucher au code Python.

Voici le code complet, intégrant **votre méthode** load_style **exacte** pour lire le fichier assets/style.qss.

Note : Assurez-vous d'avoir créé un dossier nommé assets au même endroit que votre script Python, et d'y avoir placé votre fichier style.qss.

Python

```
import sys
import os
from PyQt6.QtWidgets import QApplication, QMainWindow, QWidget, QGridLayout, QLineEdit, QPushButton
from PyQt6.QtGui import QIcon
from PyQt6.QtCore import Qt


class Calculatrice(QMainWindow):
```

```
def __init__(self):
    super().__init__()
    self.setWindowTitle("Calculatrice Clavier")
    self.setFixedSize(300, 400)

    self.load_icon()
    self.load_style() # Appel de votre méthode pour charger le fichier QSS externe

    widget_central = QWidget()
    self.setCentralWidget(widget_central)
    self.layout = QGridLayout()
    widget_central.setLayout(self.layout)

    self.creer_interface()

def creer_interface(self):
    self.ecran = QLineEdit()
    self.ecran.setFixedHeight(60)
    self.ecran.setAlignment(Qt.AlignmentFlag.AlignRight)
    self.ecran.setReadOnly(True)
    self.layout.addWidget(self.ecran, 0, 0, 1, 4)
```

```
boutons = [
    ('7', 1, 0), ('8', 1, 1), ('9', 1, 2), ('/', 1, 3),
    ('4', 2, 0), ('5', 2, 1), ('6', 2, 2), ('*', 2, 3),
    ('1', 3, 0), ('2', 3, 1), ('3', 3, 2), ('-', 3, 3),
    ('C', 4, 0), ('0', 4, 1), ('=', 4, 2), ('+', 4, 3)
]

for texte, ligne, colonne in boutons:
    bouton = QPushButton(texte)
    bouton.setFixedSize(60, 60)

    # Attribution des classes pour que le fichier QSS puisse les styliser
    if texte in ['/', '*', '-', '+', '=']:
        bouton.setProperty("class", "operateur")
    elif texte == 'C':
        bouton.setProperty("class", "effacer")
    else:
        bouton.setProperty("class", "chiffre")

    bouton.clicked.connect(self.action_bouton)
```

```
        self.layout.addWidget(bouton, ligne, colonne)

# --- LOGIQUE DES BOUTONS (Souris) ---
def action_bouton(self):
    valeur = self.sender().text()
    if valeur == 'C':
        self.ecran.clear()
    elif valeur == '=':
        self.calculer_resultat()
    else:
        self.ajouter_caractere(valeur)

# --- LOGIQUE DU CLAVIER ---
def keyPressEvent(self, event):
    touche = event.text()

    # 1. Chiffres et opérateurs (+, -, *, /)
    if touche in "0123456789+-*/":
        self.ajouter_caractere(touche)

    # 2. Touches Entrée (Clavier principal et Pavé numérique)
```

```
    elif event.key() in (Qt.Key.Key_Enter, Qt.Key.Key_Return):
        self.calculer_resultat()

    # 3. Touche Retour Arrière (Backspace) pour corriger
    elif event.key() == Qt.Key.Key_Backspace:
        texte_actuel = self.ecran.text()
        self.ecran.setText(texte_actuel[:-1]) # Enlève le dernier caractère

    # 4. Touche Échap (Escape) pour tout effacer (équivalent du 'C')
    elif event.key() == Qt.Key.Key_Escape:
        self.ecran.clear()

# --- FONCTIONS UTILITAIRES ---
def ajouter_caractere(self, caractere):
    texte_actuel = self.ecran.text()
    if texte_actuel == "Erreur":
        texte_actuel = ""
    self.ecran.setText(texte_actuel + caractere)

def calculer_resultat(self):
    try:
```

```
        # On vérifie que l'écran n'est pas vide avant de calculer
        if self.ecran.text():
            resultat = eval(self.ecran.text())
            self.ecran.setText(str(resultat))
    except Exception:
        self.ecran.setText("Erreur")

# ----------------------
# Charger l’icône depuis le fichier
# ----------------------
def load_icon(self):
    icon_path = os.path.join(os.path.dirname(__file__), "assets", "images", "favicon.ico")
    if os.path.exists(icon_path):
        self.setWindowIcon(QIcon(icon_path))
    else:
        print("Icône introuvable :", icon_path)

# ----------------------
# Charger le style QSS
# ----------------------
def load_style(self):
```

```
        style_path = os.path.join(os.path.dirname(__file__), "assets", "style.qss")
        if os.path.exists(style_path):
            with open(style_path, "r", encoding="utf-8") as f: # Ajout de l'encodage utf-8
                self.setStyleSheet(f.read())
        else:
            print("Fichier style introuvable :", style_path)


if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = Calculatrice()
    fenetre.show()
    sys.exit(app.exec())
```

Un rappel de l'arborescence requise :

Pour que votre code trouve bien les fichiers, l'organisation de vos dossiers doit ressembler exactement à ceci :

- 📂 Votre_Dossier_Projet/
  - 📄 main.py (le code ci-dessus)
  - 📂 assets/
    - 📄 style.qss (votre fichier de style contenant les classes .chiffre, .operateur, etc.)
    - 📂 images/
      - 📄 favicon.ico

Puisque votre structure de fichiers est maintenant bien définie avec des ressources externes (le dossier assets), voulez-vous que je vous explique comment utiliser **PyInstaller** pour empaqueter tout cela (le script ET le dossier assets) dans un seul fichier .exe ?

Vous avez dit

Quel est le code du fichier style.qss ?

Gemini a dit

Voici le code exact que vous devez placer à l'intérieur de votre fichier style.qss.

Il utilise les "classes" (chiffre, operateur, effacer) que nous avons définies dans le script Python avec setProperty() pour appliquer des couleurs distinctes à chaque type de bouton.

CSS

```
/* --- Fond de la fenêtre principale --- */

QWidget {

   background-color: #000000;

}


/* --- Écran d'affichage (QLineEdit) --- */

QLineEdit {

   background-color: transparent;

   border: none;

   font-size: 55px;

   color: #ffffff;

   padding-right: 15px;

   margin-bottom: 10px;
```

```
}

/* --- Style de base pour TOUS les boutons --- */
QPushButton {
    font-size: 26px;
    font-weight: bold;
    border: none;
    border-radius: 30px; /* Rend les boutons de 60x60 parfaitement ronds */
    color: #ffffff;
}

/* --- Boutons Chiffres (0 à 9) --- */
QPushButton[class="chiffre"] {
    background-color: #333333;
}
QPushButton[class="chiffre"]:hover {
    background-color: #555555;
}
QPushButton[class="chiffre"]:pressed {
    background-color: #777777;
}
```

```
/* --- Boutons Opérateurs (/, *, -, +, =) --- */
QPushButton[class="operateur"] {
    background-color: #ff9f0a;
}
QPushButton[class="operateur"]:hover {
    background-color: #ffb340;
}
QPushButton[class="operateur"]:pressed {
    background-color: #cc7f08;
}

/* --- Bouton Effacer ("C") --- */
QPushButton[class="effacer"] {
    background-color: #ff3b30;
}
QPushButton[class="effacer"]:hover {
    background-color: #ff6961;
}
QPushButton[class="effacer"]:pressed {
    background-color: #cc2f26;
```

}

Il vous suffit de sauvegarder ce texte dans le fichier assets/style.qss (avec l'encodage UTF-8, par défaut sur la plupart des éditeurs comme VS Code).

Maintenant que vous avez le script Python fonctionnel et le dossier assets complet (style + icône), souhaitez-vous que je vous donne la commande exacte pour compiler tout cela en un seul fichier exécutable .exe avec PyInstaller ?

Vous avez dit

oui

Gemini a dit

Parfait ! Créer un exécutable (un fichier .exe) permet à n'importe qui d'utiliser votre calculatrice sur Windows sans avoir besoin d'installer Python, PyQt6, ou même de voir votre code source.

Pour cela, nous allons utiliser l'outil incontournable : **PyInstaller**.

Voici la marche à suivre étape par étape.

Étape 1 : Installer PyInstaller

Ouvrez votre terminal (ou invite de commandes) dans le dossier de votre projet et tapez :

Bash

```
pip install pyinstaller
```

Étape 2 : Lancer la compilation

Dans ce même terminal, assurez-vous d'être dans le dossier où se trouve votre fichier main.py (ou le nom que vous lui avez donné).

Copiez-collez cette commande exacte :

Bash

```
pyinstaller --noconsole --onefile --icon="assets/images/favicon.ico" --add-data "assets;assets" main.py
```

Que fait cette commande ?

- --noconsole (ou -w) : Cache la fenêtre noire du terminal en arrière-plan (indispensable pour une application graphique pure).

- --onefile (ou -F) : Empaquette tout (Python, PyQt6, votre script) dans un seul et unique fichier .exe.
- --icon="..." : Attribue votre icône directement au fichier .exe pour qu'il soit beau sur le bureau Windows.
- --add-data "assets;assets" : Dit à PyInstaller de prendre votre dossier assets (qui contient le style et l'icône) et de l'inclure à l'intérieur de l'exécutable. *(Note: sur Mac/Linux, on utilise : au lieu de ;)*.

Étape 3 : Récupérer votre fichier .exe

Une fois que le terminal a terminé (cela peut prendre 1 à 2 minutes la première fois), PyInstaller aura créé plusieurs nouveaux dossiers dans votre projet.

1. Allez dans le nouveau dossier nommé dist.
2. Vous y trouverez votre fichier main.exe (ou le nom de votre script).
3. C'est ce fichier (et lui seul) que vous pouvez envoyer à vos amis ! Vous pouvez supprimer les dossiers build et le fichier main.spec, ils ne servent qu'à PyInstaller.

⚠️ **Point très important (Le secret des chemins PyInstaller)**

Quand PyInstaller crée un fichier --onefile, il décompresse secrètement votre dossier assets dans un dossier temporaire de Windows au moment où vous double-cliquez sur le .exe.

Si jamais votre .exe se lance mais que **le style QSS ou l'icône a disparu**, c'est parce que __file__ ne pointe plus au bon endroit. Pour rendre votre code 100% "PyInstaller-proof", on utilise généralement cette petite fonction magique au tout début du script :

Python

```
def chemin_ressource(chemin_relatif):
    """ Récupère le chemin absolu, que ce soit pour le dev ou pour PyInstaller """
    try:
        # PyInstaller crée un dossier temporaire et stocke le chemin dans _MEIPASS
        chemin_base = sys._MEIPASS
```

```
    except Exception:
        chemin_base = os.path.abspath(".")


    return os.path.join(chemin_base, chemin_relatif)
```

Et ensuite, au lieu de faire os.path.join(os.path.dirname(__file__), "assets", "style.qss"), vous utiliseriez simplement : chemin_ressource("assets/style.qss").

Testez d'abord la compilation avec votre code actuel. Si le style s'affiche bien dans le .exe, vous n'avez rien à changer !

Souhaitez-vous que je vous intègre cette fonction de sécurité dans votre code final pour être certain que le .exe fonctionnera à 100% sur n'importe quel ordinateur ?

Vous avez dit

oui

Gemini a dit

C'est la meilleure approche pour garantir que votre application fonctionne partout, sans mauvaise surprise !

Voici le code final, complet et "PyInstaller-proof". J'ai ajouté la fonction chemin_ressource en haut du fichier et je l'ai intégrée dans load_icon et load_style.

Le code Python final (main.py**)**

Vous pouvez copier-coller l'intégralité de ce code pour remplacer le vôtre :

Python

```
import sys

import os

from PyQt6.QtWidgets import QApplication, QMainWindow, QWidget, QGridLayout, QLineEdit, QPushButton

from PyQt6.QtGui import QIcon

from PyQt6.QtCore import Qt
```

```
# --- FONCTION MAGIQUE POUR PYINSTALLER ---
def chemin_ressource(chemin_relatif):
    """ Obtenir le chemin absolu vers la ressource, fonctionne pour le dev et pour PyInstaller """
    try:
        # PyInstaller crée un dossier temporaire et stocke le chemin dans _MEIPASS
        chemin_base = sys._MEIPASS
    except Exception:
        chemin_base = os.path.abspath(".")

    return os.path.join(chemin_base, chemin_relatif)
# ----------------------------------------

class Calculatrice(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Calculatrice Clavier")
        self.setFixedSize(300, 400)

        self.load_icon()
        self.load_style()
```

```
    widget_central = QWidget()
    self.setCentralWidget(widget_central)
    self.layout = QGridLayout()
    widget_central.setLayout(self.layout)

    self.creer_interface()

def creer_interface(self):
    self.ecran = QLineEdit()
    self.ecran.setFixedHeight(60)
    self.ecran.setAlignment(Qt.AlignmentFlag.AlignRight)
    self.ecran.setReadOnly(True)
    self.layout.addWidget(self.ecran, 0, 0, 1, 4)

    boutons = [
        ('7', 1, 0), ('8', 1, 1), ('9', 1, 2), ('/', 1, 3),
        ('4', 2, 0), ('5', 2, 1), ('6', 2, 2), ('*', 2, 3),
        ('1', 3, 0), ('2', 3, 1), ('3', 3, 2), ('-', 3, 3),
        ('C', 4, 0), ('0', 4, 1), ('=', 4, 2), ('+', 4, 3)
    ]
```

```
    for texte, ligne, colonne in boutons:
        bouton = QPushButton(texte)
        bouton.setFixedSize(60, 60)

        if texte in ['/', '*', '-', '+', '=']:
            bouton.setProperty("class", "operateur")
        elif texte == 'C':
            bouton.setProperty("class", "effacer")
        else:
            bouton.setProperty("class", "chiffre")

        bouton.clicked.connect(self.action_bouton)
        self.layout.addWidget(bouton, ligne, colonne)

def action_bouton(self):
    valeur = self.sender().text()
    if valeur == 'C':
        self.ecran.clear()
    elif valeur == '=':
        self.calculer_resultat()
```

```
    else:
        self.ajouter_caractere(valeur)

def keyPressEvent(self, event):
    touche = event.text()

    if touche in "0123456789+-*/":
        self.ajouter_caractere(touche)

    elif event.key() in (Qt.Key.Key_Enter, Qt.Key.Key_Return):
        self.calculer_resultat()

    elif event.key() == Qt.Key.Key_Backspace:
        texte_actuel = self.ecran.text()
        self.ecran.setText(texte_actuel[:-1])

    elif event.key() == Qt.Key.Key_Escape:
        self.ecran.clear()

def ajouter_caractere(self, caractere):
    texte_actuel = self.ecran.text()
```

```
    if texte_actuel == "Erreur":
        texte_actuel = ""
    self.ecran.setText(texte_actuel + caractere)

def calculer_resultat(self):
    try:
        if self.ecran.text():
            resultat = eval(self.ecran.text())
            self.ecran.setText(str(resultat))
    except Exception:
        self.ecran.setText("Erreur")

# --- MODIFIÉ POUR UTILISER chemin_ressource ---
def load_icon(self):
    # On remplace os.path.dirname(__file__) par la fonction magique
    icon_path = chemin_ressource(os.path.join("assets", "images", "favicon.ico"))
    if os.path.exists(icon_path):
        self.setWindowIcon(QIcon(icon_path))
    else:
        print("Icône introuvable :", icon_path)
```

```
    # --- MODIFIÉ POUR UTILISER chemin_ressource ---
    def load_style(self):
        # On remplace os.path.dirname(__file__) par la fonction magique
        style_path = chemin_ressource(os.path.join("assets", "style.qss"))
        if os.path.exists(style_path):
            with open(style_path, "r", encoding="utf-8") as f:
                self.setStyleSheet(f.read())
        else:
            print("Fichier style introuvable :", style_path)


if __name__ == "__main__":
    app = QApplication(sys.argv)
    fenetre = Calculatrice()
    fenetre.show()
    sys.exit(app.exec())
```

Le rappel de la commande PyInstaller

Désormais, vous pouvez générer votre exécutable sans aucune crainte avec cette commande dans votre terminal :

Bash

```
pyinstaller --noconsole --onefile --icon="assets/images/favicon.ico" --add-data "assets;assets" main.py
```

(Si vous êtes sur Mac ou Linux, remplacez assets;assets par assets:assets).

Votre calculatrice est maintenant un projet professionnel achevé, du code à l'interface, jusqu'au déploiement ! Y a-t-il une autre fonctionnalité (comme l'historique des calculs ou l'ajout de sons) que vous aimeriez explorer pour un prochain projet ?

Vous avez dit

PS D:\PYTHON DJANGO HOPES\LOGICIEL PyQt COURS\EXO\BOUTON_INCREMENTE> & "D:/PYTHON DJANGO HOPES/LOGICIEL PyQt COURS/EXO/BOUTON_INCREMENTE/env/Scripts/Activate.ps1"

S/EXO/BOUTON_INCREMENTE/env/Scripts/python.exe" "d:/PYTHON DJANGO HOPES/LOGICIEL PyQt COURS/EXO/BOUTON_INCREMENTE/calculatrice_simple_clavier/main.py"

Fichier style introuvable : D:\PYTHON DJANGO HOPES\LOGICIEL PyQt COURS\EXO\BOUTON_INCREMENTE\assets\style.qss

(env) PS D:\PYTHON DJANGO HOPES\LOGICIEL PyQt COURS\EXO\BOUTON_INCREMENTE> & "d:\PYTHON DJANGO HOPES\LOGICIEL PyQt COURS\EXO\BOUTON_INCREMENTE\env\Scripts\Activate.ps1"

(env) PS D:\PYTHON DJANGO HOPES\LOGICIEL PyQt COURS\EXO\BOUTON_INCREMENTE> & "D:/PYTHON DJANGO HOPES/LOGICIEL PyQt COURlatrice_simple_clavier/main.py"

Icône introuvable : D:\PYTHON DJANGO HOPES\LOGICIEL PyQt COURS\EXO\BOUTON_INCREMENTE\assets\images\favicon.ico

Fichier style introuvable : D:\PYTHON DJANGO HOPES\LOGICIEL PyQt COURS\EXO\BOUTON_INCREMENTE\assets\style.qss

(env) PS D:\PYTHON DJANGO HOPES\LOGICIEL PyQt COURS\EXO\BOUTON_INCREMENTE> & "D:/PYTHON DJANGO HOPES/LOGICIEL PyQt COURS/EXO/BOUTON_INCREMENTE/env/Scripts/python.exe" "d:/PYTHON DJANGO HOPES/LOGICIEL PyQt COURS/EXO/BOUTON_INCREMENTE/calculatrice_simple_clavier/main.py"

Icône introuvable : D:\PYTHON DJANGO HOPES\LOGICIEL PyQt COURS\EXO\BOUTON_INCREMENTE\assets\images\favicon.ico

Fichier style introuvable : D:\PYTHON DJANGO HOPES\LOGICIEL PyQt COURS\EXO\BOUTON_INCREMENTE\assets\style.qss

(env) PS D:\PYTHON DJANGO HOPES\LOGICIEL PyQt COURS\EXO\BOUTON_INCREMENTE>

Gemini a dit

L'erreur affichée dans votre terminal est très claire et c'est un problème classique en programmation : **un souci de répertoire de travail (Working Directory).**

Regardez attentivement l'erreur : Votre terminal est ouvert dans le dossier .../EXO/BOUTON_INCREMENTE>. Mais votre fichier Python se trouve dans le sous-dossier .../EXO/BOUTON_INCREMENTE/calculatrice_simple_clavier/main.py.

Pourquoi ça plante ?

La fonction chemin_ressource que nous avons utilisée contient la ligne os.path.abspath("."). Le "." veut dire "le dossier dans lequel le terminal est actuellement ouvert". Le programme cherche donc le dossier assets dans BOUTON_INCREMENTE/assets au lieu de chercher dans BOUTON_INCREMENTE/calculatrice_simple_clavier/assets.

La solution (La plus robuste)

Nous allons corriger la fonction magique chemin_ressource pour qu'elle utilise **l'emplacement réel du fichier** main.py plutôt que l'emplacement du terminal. C'est la méthode la plus sûre.

Remplacez simplement la fonction chemin_ressource tout en haut de votre code par celle-ci :

Python

```
# --- FONCTION MAGIQUE POUR PYINSTALLER (CORRIGÉE) ---

def chemin_ressource(chemin_relatif):

    """ Obtenir le chemin absolu vers la ressource, fonctionne pour le dev et pour PyInstaller """

    try:

        # PyInstaller crée un dossier temporaire et stocke le chemin dans _MEIPASS

        chemin_base = sys._MEIPASS

    except Exception:

        # Au lieu de ".", on prend le dossier exact où se trouve ce fichier Python (__file__)

        chemin_base = os.path.dirname(os.path.abspath(__file__))


    return os.path.join(chemin_base, chemin_relatif)
```

Ce qui change : J'ai remplacé os.path.abspath(".") par os.path.dirname(os.path.abspath(__file__)). Ainsi, peu importe depuis quel dossier vous lancez la commande dans le terminal, Python saura toujours que le dossier assets se trouve juste à côté de main.py.

L'alternative (Moins pratique)

Si vous ne vouliez pas changer le code, la solution aurait été de vous déplacer dans le bon dossier directement dans votre terminal avant de lancer le script :

PowerShell

cd calculatrice_simple_clavier

python main.py

Mais je vous conseille vivement d'utiliser la modification de code ci-dessus, cela vous évitera bien des soucis à l'avenir, surtout lors de la création de votre .exe !

Est-ce que l'interface s'affiche bien avec toutes ses couleurs maintenant ?

Vous avez dit

oui

Gemini a dit

Super ! C'est une excellente nouvelle. 🎉

Maintenant que votre script fonctionne à la perfection et que les chemins vers vos fichiers (style et icône) sont devenus 100 % robustes, nous pouvons passer à la consécration : **la création du fichier exécutable (**.exe**)**.

Voici les étapes exactes à suivre dans votre terminal PowerShell pour compiler votre calculatrice :

1. Placez-vous dans le bon dossier

Puisque votre terminal s'ouvre souvent dans le dossier parent BOUTON_INCREMENTE, il faut d'abord dire au terminal d'entrer dans le dossier du projet. Tapez ceci et appuyez sur Entrée :

PowerShell

```
cd "calculatrice_simple_clavier"
```

(Vérifiez que la ligne de votre terminal se termine bien par calculatrice_simple_clavier>).

2. Lancez la compilation (CREATION DU FICHIER EXECUTABLE WINDOWS .exe) avec PyInstaller

Assurez-vous d'avoir installé l'outil (si ce n'est pas déjà fait) avec pip install pyinstaller. Ensuite, copiez-collez cette commande magique et appuyez sur Entrée :

PowerShell

```
pyinstaller --noconsole --onefile --icon="assets/images/favicon.ico" --add-data "assets;assets" main.py
```

3. Récupérez votre application !

Une fois que le terminal a fini de mouliner (cela prend généralement une petite minute), regardez dans votre dossier calculatrice_simple_clavier. PyInstaller a créé plusieurs choses :

- Un dossier dist : **C'est là que se trouve votre fichier** main.exe **!**
- Un dossier build et un fichier main.spec : Vous pouvez les supprimer en toute sécurité, ils ne servaient qu'à la fabrication.

Vous pouvez maintenant renommer votre main.exe en Calculatrice.exe, le glisser sur votre bureau et le lancer d'un simple double-clic. Il fonctionnera sans afficher de console noire derrière, et gardera son beau design QSS.

Considérez-vous ce projet comme parfaitement achevé ou souhaitez-vous explorer d'autres choses avec PyQt6 ?

Gemini est une IA et peut se tromper.